E2 DAMA-DMBOK applied to talent data

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track E: Data protection and governance 2

E2 DAMA-DMBOK applied to talent data 3

E2-L1 The DAMA wheel, eleven knowledge areas around governance 3

E2-L2 A maturity baseline you can defend 7

E2-L3 Governance at the hub, decision rights, policies and escalation 11

E2-L4 People master data, match rules and survivorship 14

E2-L5 A RACI and a business glossary for talent data 19

Track E best-practice checklist 24

Appendix: solutions and answer keys 25

Track E: Data protection and governance

The law and the frameworks that govern people data, for the engineer who builds the platform.

E2 DAMA-DMBOK applied to talent data

Roadmap: EY Talent Data Strategy role; Current work. The eleven knowledge areas, governance at the hub, and an evidence-based maturity baseline across talent domains.

E2-L1 The DAMA wheel, eleven knowledge areas around governance

Concept

The **DAMA-DMBOK** (Data Management Body of Knowledge, second edition, revised in 2024) gives data teams a shared map. It splits data management into **eleven knowledge areas**, drawn as a wheel.

Data governance sits at the hub. It decides and oversees how the other ten are run. DMBOK2 defines it as the exercise of authority, control and shared decision-making (planning, monitoring and enforcement) over the management of data assets.

The ten spokes, with a talent example for each:

Knowledge area	Talent example
Data architecture	Bronze, silver and gold layers for HR data
Data modelling and design	The person, position and organisation entities
Data storage and operations	Lakehouse tables, backups, retention jobs
Data security	Masking salary and health fields
Data integration and interoperability	Feeds from the HR system and the learning platform
Document and content management	Contracts, review forms, scanned certificates
Reference and master data	One golden record per person, job codes, grades
Data warehousing and business intelligence	Headcount and attrition reports
Metadata management	Glossary entries, lineage, owners
Data quality	Rules for missing managers or duplicate employee IDs

Two pairs get confused. **Reference data** is a controlled list of codes, such as grades or country codes. **Master data** is the agreed record of a core entity, such as the person. **Metadata** describes data: definitions, owners, lineage. **Master data** is the data itself.

The wheel is a checklist, not an org chart. A talent platform touches all eleven areas. When something breaks, name the area first, then the owner. A duplicate employee is a master data problem, not a reporting bug.

Why it matters

A firm's headcount report disagreed with payroll by 312 people. The BI team spent six weeks tuning report filters. The cause was in master data: contractors re-hired through an agency got a second employee ID, and nothing matched them to their first record. Naming the knowledge area on day one would have sent the problem to the master data owner.

Worked example: weak code

```
# Issue log, talent platform
- issue: headcount differs from payroll by 312
  owner: BI team
  fix: adjust the report filter
```

1. **Line 3.** The report shows the symptom. The cause sits in another knowledge area.
2. **Line 4.** Fixing the symptom hides the duplicate records from every other consumer.

Worked example: strong code

```
# Issue log, talent platform
- issue: headcount differs from payroll by 312
  knowledge_area: reference and master data
  root_cause: re-hired contractors get a second employee ID
  owner: people master data owner
  fix: match rule on national insurance number and date of birth, then
   merge
```

1. **Line 3.** Name the area first. It points at the right owner.
2. **Line 5.** Governance names an owner for each area and domain.

Name the knowledge area before you name the fix. Governance at the hub assigns an owner to each area, so problems reach the person who can solve them.

Recap card

- **DAMA wheel** Data governance at the hub, ten knowledge areas around it. DMBOK2, revised in 2024.
- **DMBOK2 definition of data governance** The exercise of authority, control and shared decision-making (planning, monitoring and enforcement) over the management of data assets.
- **Reference data, master data, metadata** Reference: lists of codes. Master: the agreed record of a core entity. Metadata: data about data.
- **First move on any data problem** Name the knowledge area, then its owner, then the fix.

Practice

Exercise 1. Multiple choice (level 1)

Which knowledge area sits at the hub of the DAMA wheel?

- a) Data architecture
- b) Data governance
- c) Data quality
- d) Metadata management

Exercise 2. Multiple choice (level 2)

Re-hired contractors get a second employee ID, so headcount is overstated. Which knowledge area owns the root cause?

- a) Data warehousing and business intelligence
- b) Reference and master data
- c) Document and content management
- d) Data storage and operations

Why?

- r1) Master data keeps one agreed record per core entity, such as a person.
- r2) Reference data controls the format of employee IDs.

Exercise 3. Fill the gap (level 2)

Complete the two definitions that are easy to confuse.

```
_____(1) data: a controlled list of codes, such as grades or country codes
_____(2): data about data, such as definitions, owners and lineage
```

Exercise 4. Architecture design (level 3)

Send each talent data problem to its knowledge area.

Options: Data security, Data quality, Data warehousing and business intelligence.

1. Analysts can read salaries they should not see
2. Two teams define 'active employee' differently
3. 12% of records have no manager

Exercise 5. Architecture design (level 4)

You start a new governance role on a talent platform. Choose how to use the DAMA wheel in your first month.

Options: Map all eleven areas to the six talent domains, then pick where to start, Start with the area you know best and add others later.

1. Scope
2. Owners
3. How the wheel is used

Sources

- DAMA-DMBOK2 has eleven knowledge areas with data governance at the centre of the wheel: data architecture, data modelling and design, data storage and operations, data security, data integration and interoperability, document and content management, reference and master data, data warehousing and business intelligence, metadata management and data quality. (verified: DAMA-DMBOK2, checked in the printed book by the course owner, 7 November 2026: chapter 1, the DAMA Wheel figure.)
- The 2024 revision of DMBOK2 kept the DAMA Wheel and its eleven knowledge areas. (verified: Confirmed by the course owner from the revised edition, 7 November 2026.)
- DMBOK2 chapter 3, section 1, defines data governance as 'the exercise of authority, control, and shared decision-making (planning, monitoring, and enforcement) over the management of data assets' (page 77 in the copy checked). (verified: DAMA-DMBOK2, checked in the printed book by the course owner, 7 November 2026.)
- Earlier summaries quoted the definition without 'shared decision-making' and gave page 67. (search-only: Government of Canada GCcollab wiki

'Data Governance' citing DMBOK2, and Perficient (2012) citing the DAMA Dictionary, seen through search results, November 2026.)

E2-L2 A maturity baseline you can defend

Concept

A **maturity baseline** records how well each knowledge area is run today, so you can set targets and show progress. DMBOK2 describes a data management maturity assessment. Sources differ on the exact level names, so agree one scale with your stakeholders and write it down. A common version runs from 0 to 5:

- **0** no capability
- **1** initial: ad hoc, depends on individuals
- **2** repeatable: some process, applied unevenly
- **3** defined: a documented standard, followed across teams
- **4** managed: measured, with targets and reviews
- **5** optimised: improved continuously from the measures

Each level builds on the last.

Score a **grid**, not a single number: knowledge areas down the side, talent domains across the top. A talent platform has six domains: workforce and organisation, recruitment, learning, performance, mobility and skills. Data quality can be at 3 for workforce data and at 1 for skills.

Three rules make a baseline you can defend.

1. **Evidence for every score.** A score without a document, a metric or a log is an opinion. "Level 3: quality rules documented in DQ-STD-02 and run nightly" survives a challenge.
2. **Report the weakest area, not the mean.** An average of 3 can hide a 0 in data security. One weak area can sink a domain.
3. **Integers only.** A level is a stage you have reached. A score of 2.5 says you have not agreed which stage you are at.

The baseline is a snapshot. Rescore at fixed intervals with the same scale, so changes mean progress, not a new yardstick.

Why it matters

A firm reported its talent data maturity as 3.1 out of 5 to its board and won budget for an AI pilot. The pilot stalled in week two: nobody owned access to

performance data, which scored 0 for data security in the performance domain. The average had hidden it. A grid reporting the weakest area per domain would have put security ahead of the pilot.

Worked example: weak code

```
# Maturity baseline, talent data
overall: 3.1
data_security: 2.5
evidence: workshop with the HR leadership team
```

1. **Line 2.** One mean for the whole platform hides the weakest area.
2. **Line 3.** A fractional level means the stage was never agreed.
3. **Line 4.** Opinions, not artefacts. Nothing an auditor can check.

Worked example: strong code

```
# Maturity baseline, talent data, scale DMMA-0-5 v1
performance:
  data_security: {score: 0, evidence: "no access model; analysts read raw reviews"}
  data_quality: {score: 2, evidence: "rules in DQ-PERF-01, run by hand monthly"}
  weakest: {area: data_security, score: 0}
workforce:
  data_security: {score: 3, evidence: "role model SEC-STD-01, quarterly access review log"}
  weakest: {area: metadata, score: 1}
```

1. **Line 1.** The scale is named and versioned, so rescoring uses the same yardstick.
2. **Line 3.** Every score cites evidence.
3. **Line 5.** Each domain reports its weakest area, not its average.

Score a grid of knowledge areas by talent domain, in whole levels, with evidence for each score, and report the weakest area per domain.

Recap card

- **A common 0 to 5 maturity scale** 0 none, 1 initial, 2 repeatable, 3 defined, 4 managed, 5 optimised. Agree and version the names with stakeholders.
- **The baseline grid** Knowledge areas by talent domain: workforce, recruitment, learning, performance, mobility, skills.
- **Summarise a domain by** Its weakest area and score. A mean hides gaps.

- **A defensible score** A whole level with cited evidence: a document, a metric or a log.

Practice

Exercise 1. Multiple choice (level 1)

Data quality rules for the learning domain are documented in a standard and followed by every team, but nobody measures them yet. Which level fits, on the 0 to 5 scale in this lesson?

- a) 1 initial
- b) 3 defined
- c) 4 managed
- d) 5 optimised

Exercise 2. Multiple choice (level 2)

The performance domain scores 4, 4, 3, 4 and 0 across five knowledge areas. What should the baseline report for the domain?

- a) 3, the mean
- b) 4, the most common score
- c) 0, with the area that scored it
- d) Nothing until every area reaches 3

Why?

- r1) One weak area can sink a domain, and a mean hides it.
- r2) The lowest score is always data security.

Exercise 3. Put in order (level 2)

Put the steps of building a maturity baseline in order.

- Agree and version the scale with stakeholders
- Score each cell in whole levels, citing evidence
- Lay out the grid: knowledge areas by talent domain
- Report the weakest area per domain
- Rescore at a fixed interval on the same scale

Exercise 4. Write the function (level 3)

Write `weakest_by_domain(scores: dict[str, dict[str, int]]) -> dict[str, tuple[str, int]]`. `scores` maps each talent domain to a dict of knowledge

area to score. Return, for each domain, the weakest area and its score. When two areas tie, pick the one that comes first alphabetically.

Raise `ValueError` if any score is not a whole number from 0 to 5, or if a domain has no scores. A `bool` does not count as a whole number.

What you need to know.

`isinstance(value, int)` checks a type. In Python, `True` is also an `int`, so check for `bool` first:

```
isinstance(3, int)       # True
isinstance(2.5, int)     # False
isinstance(True, bool)   # True, and isinstance(True, int) is True too
```

`min` takes a `key` function. A tuple key compares item by item: `min([("b", 1), ("a", 1)], key=lambda p: (p[1], p[0]))` returns `("a", 1)`.

Your code must pass these checks:

- Return the lowest-scoring area and its score for each domain.
- When areas tie, pick the one that comes first alphabetically.
- Raise ValueError for a score outside 0 to 5, a fraction, a bool, or a domain with no scores.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
MIN_LEVEL = ...

# Module constant.
MAX_LEVEL = ...

# The weakest knowledge area and its score for each talent domain.
def weakest_by_domain(
    scores: dict[str, dict[str, int]],
) -> dict[str, tuple[str, int]]: ...
```

Starter code:

```
def weakest_by_domain(scores):
    return {domain: min(areas.items()) for domain, areas in
    scores.items()}
```

Exercise 5. Architecture design (level 4)

Plan how you present the first talent data maturity baseline to HR leadership.

Options: A grid of areas by domain, with the weakest area per domain highlighted, One overall maturity score out of 5.

1. Headline

2. Evidence
3. What the baseline leads to

Sources

- DMBOK2 chapter 15, Data Management Maturity Assessment, uses six levels: 0 no capability, 1 initial or ad hoc, 2 repeatable, 3 defined, 4 managed, 5 optimised. It starts at 0, unlike CMM models that start at 1. (verified: DAMA-DMBOK2, checked in the printed book by the course owner, 7 November 2026.)
- The six talent domains (workforce and organisation, recruitment, learning, performance, mobility, skills) come from the role brief, not from DAMA. (verified: content/roles/ey-talent.yaml, the job description in the brief.)

E2-L3 Governance at the hub, decision rights, policies and escalation

Concept

Governance and management are different jobs. **Data governance** decides and oversees: who may decide what, which rules apply, and whether they are followed. **Data management** does the work: builds the pipelines, runs the quality checks, grants the access. DMBOK2 frames governance as setting decision rights over data. A platform team that both writes the rules and marks its own work has no oversight.

Start with **decision rights**. For each kind of decision, name who decides:

- what a term means, such as "active employee" or "regretted leaver"
- who may see a field, such as salary or a performance rating
- what quality threshold a dataset must meet before it is published
- whether a new use of data goes ahead

Then write the rules at three levels:

- A **policy** says what must be true, and why. "Access to performance data follows the principle of least privilege."
- A **standard** makes it measurable. "Ratings are masked for every role except HR business partners and the employee's own line chain."
- A **procedure** says how. "Request access through the form. The data owner approves within five working days."

Policies change rarely and need senior sign-off. Standards change when the platform does. Procedures change often.

Finally, an **escalation path**. Most issues end with a data steward. Those that cross domains, or set precedent, go to the data owner, then to a governance council. Each decision is written down in a decision log (lesson B18-L3), so the next person finds the reason, not only the rule.

GOV-02 in your EY role plan turns these rights into a RACI.

Why it matters

Two directors argued for a quarter over whether contractors counted in "headcount". The data team changed the dashboard each time one of them complained. Nobody had the right to decide, so the definition flipped four times and trust in every people report fell. A named decision owner and one log entry settled it in a week.

Worked example: weak code

```
# Governance, talent platform
rules: the data platform team decides and enforces everything
access_rule: be careful with sensitive data
disputes: raise in the weekly team meeting
```

1. **Line 2.** The same team writes the rules and marks its own work. No oversight.
2. **Line 3.** Not measurable. Nobody can test it.
3. **Line 4.** No owner, no record, so the same dispute comes back.

Worked example: strong code

```
# Governance, talent platform
decision_rights:
  definitions: {decides: data owner per domain, advises: data steward}
  access_to_performance_data: {decides: performance data owner}
policy: POL-07 least privilege for performance data
standard: STD-07 ratings masked except HR business partners and line chain
procedure: PRC-07 request form, owner approves within 5 working days
escalation: [data steward, data owner, governance council]
decisions: logged in DEC-LOG with the reason and the date
```

1. **Line 3.** One named decider per kind of decision.
2. **Line 6.** The standard makes the policy testable.
3. **Line 8.** A fixed path, so disputes end with a decision.

Separate deciding from doing. Name a decider for each kind of decision, write rules as policy, standard and procedure, and log each decision with its reason.

Recap card

- **Governance against management** Governance decides and oversees. Management does the work.
- **Decision rights to name first** Definitions, access to sensitive fields, quality thresholds, and new uses of data.
- **Policy, standard, procedure** Policy: what and why. Standard: measurable. Procedure: how. Each changes at its own pace.
- **Escalation path** Data steward, data owner, governance council. Log every decision with its reason.

Practice

Exercise 1. Multiple choice (level 1)

Which of these is a data governance activity rather than a data management one?

- a) Running the nightly quality checks
- b) Deciding who may approve access to salary data
- c) Building the silver layer for learning records
- d) Granting a role in a workspace

Exercise 2. Multiple choice (level 2)

Which statement is a standard, not a policy or a procedure?

- a) Access to performance data follows least privilege.
- b) Ratings are masked for every role except HR business partners and the line chain.
- c) Request access through the form, and the owner approves within five working days.
- d) Be careful with sensitive data.

Why?

- r1) A standard makes a policy measurable, so you can test whether it holds.
- r2) A standard says who signs it off.

Exercise 3. Fill the gap (level 2)

Complete the three levels of rules.

```
_____(1): what must be true, and why. Changes rarely, senior sign-off.
_____(2): makes it measurable. Changes when the platform does.
```

```
3 _____(3): how to do it. Changes often.
```

Exercise 4. Put in order (level 3)

A dispute over what "regretted leaver" means affects two domains. Put the escalation steps in order.

- The data owners of both domains try to agree
- Update the glossary entry and tell the report owners
- Record the decision and its reason in the decision log
- The data steward gathers the definitions in use and their impact
- Without agreement, the governance council decides

Exercise 5. Architecture design (level 4)

Set up governance for the talent platform's first domain, performance data.

Options: The performance data owner, a named HR leader, advised by a steward, The data platform team, because it builds the tables.

1. Who decides what a performance rating means
2. The access rule
3. Disputes

Sources

- DMBOK2 chapter 3 separates governance, a formal system of decision rights over data with an escalation path for issues, from data management, which carries out the work. A Data Governance Council is the highest functional authority for policy and data disputes. (verified: DAMA-DMBOK2, checked in the printed book by the course owner, 7 November 2026.)
- Policy, standard and procedure as three levels of rules, and the steward, owner, council escalation path, are common governance practice, not quotations from DMBOK2. (unverified: General practice. Not attributed to a specific DMBOK2 passage.)

E2-L4 People master data, match rules and survivorship

Concept

Lesson E2-L1 named the problem: one person, two employee IDs. Master data management fixes it in two steps, and each step needs a written rule that the master data owner signs off.

Match: which records are the same person?

- A **deterministic** rule matches on exact keys, such as national insurance number plus date of birth. It is easy to explain and misses typos.
- A **probabilistic** rule scores similarity across fields, such as name, date of birth and email, and matches above a threshold. It catches more and makes some false matches.

Run both in bands. Above a high score, merge automatically. Below a low score, keep the records apart. In between, send the pair to a **data steward** queue. A false merge mixes two people's salaries and appraisals, which is worse than a missed match.

Survivorship: which value wins? Once records match, the **golden record** takes each field from one of them. Write a rule per field:

- **Source priority**: the HR system wins for legal name and grade, the learning platform for course history.
- **Most recent**: for a home address, the latest update wins.
- **Most complete**: a value beats a missing one. A blank must never overwrite a value.

DMBOK2 names two patterns. **Match-merge** builds one physical record. **Match-link** only links source records to a global ID, which is easier to reverse.

Either way, keep a **cross-reference** from the golden ID to every source ID it absorbed. Then a later delivery from any source still finds its person, and an unmerge stays possible when a steward finds a false match.

The rules are data, kept in a versioned file the owner signs off, not logic buried in a notebook.

Why it matters

A firm's master data job merged two employees called Daniel Hughes, born in the same year, because its rule matched on name and birth year only. For three weeks one Daniel saw the other's performance rating in the self-service portal. The firm reported the breach to the regulator, and rewrote the rule to need a national insurance number match before any automatic merge.

Worked example: weak code

```
# Person matching, talent platform
match: same first name, surname and birth year
merge: automatic
```

```
❸ survivorship: the last record loaded wins every field
```

1. **Line 2.** Two different people share these often in a large firm.
2. **Line 3.** No steward review for uncertain pairs, so false merges go live.
3. **Line 4.** A blank or stale field from the latest load can overwrite a correct value.

Worked example: strong code

```
 1 # Person matching, talent platform, version 3, signed off by the people
     master data owner
 2 match:
❶   auto_merge: national insurance number and date of birth agree
❷   steward_review: name, date of birth and email score 0.80 to 0.95
 5   keep_apart: score below 0.80
 6 survivorship:
❸   legal_name: {rule: source_priority, order: [hr_system, payroll,
     learning]}
 8   grade: {rule: source_priority, order: [hr_system, payroll]}
 9   home_address: {rule: most_recent}
10   any_field: never overwrite a value with a blank
❹ cross_reference: golden ID to every source ID, kept for unmerge
```

1. **Line 3.** Automatic merges only on a deterministic key.
2. **Line 4.** The uncertain band goes to a person.
3. **Line 7.** A rule per field, with the trusted source first.
4. **Line 11.** Source IDs stay findable, and a false merge can be undone.

Match in bands, with only deterministic keys merging automatically. Choose each field's value by a written rule, never let a blank win, and keep the cross-reference.

Recap card

- **Deterministic or probabilistic matching** Deterministic: exact keys, easy to explain, misses typos. Probabilistic: similarity score, catches more, makes false matches.
- **Match bands** Merge above a high score, keep apart below a low one, and send the middle band to a data steward.
- **Common survivorship rules** Source priority, most recent, most complete. A blank never overwrites a value.
- **Why keep a cross-reference?** Later deliveries find their person by source ID, and a false merge can be undone.

Practice

Exercise 1. Multiple choice (level 1)

What does a survivorship rule decide?

- a) Which value each field of the golden record takes
- b) Which records belong to the same person
- c) Who may see the golden record
- d) How long the source records are kept

Exercise 2. Multiple choice (level 2)

A probabilistic match scores 0.87 for two employee records. Your bands are: merge above 0.95, keep apart below 0.80. What happens?

- a) The pair goes to the data steward queue
- b) The records merge automatically
- c) The records stay apart and nobody reviews them
- d) The newest record replaces the older one

Why?

- r1) A score between the bands is uncertain, and a false merge mixes two people's data.
- r2) Stewards review every match, whatever its score.

Exercise 3. Fill the gap (level 2)

Complete the survivorship rules.

```
legal_name:    source _____(1), the HR system first
home_address:  most _____(2) update wins
any field:     never overwrite a value with a _____(3)
```

Exercise 4. Write the function (level 3)

Write `survive(records: list[dict], priority: list[str]) -> dict`. Each record has a `"source"` key and field values, some of which may be `None`. `priority` lists sources, most trusted first.

Return the golden record: for every field other than `"source"`, take the value from the most trusted source whose value is not `None`. Leave out a field that is `None` in every record. Raise `ValueError` if a record's source is not in `priority`.

What you need to know.

`list.index(item)` gives an item's position: `["a", "b", "c"].index("b")` is `1`. It raises `ValueError` for a missing item.

`sorted` takes a `key` function and returns a new list:

```
ranks = ["gold", "silver", "bronze"]
medals = [{"m": "bronze"}, {"m": "gold"}]
sorted(medals, key=lambda d: ranks.index(d["m"]))   # gold first
```

Test for a missing value with `value is None`, so `0` and `""` still count as values.

Your code must pass these checks:

- Take each field from the most trusted source, whatever order the records arrive in.
- A None from a trusted source must not beat a real value from a less trusted one.
- 0 is a value. Test for a missing value with `is None`.
- Leave out a field that is None in every record.
- Raise ValueError for a source that is not in the priority list.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# The golden record built from matched source records.
def survive(
    records: list[dict],
    priority: list[str],
) -> dict: ...
```

Starter code:

```
def survive(records, priority):
    golden = {}
    for record in records:
        golden.update(record)
    golden.pop("source", None)
    return golden
```

Exercise 5. Architecture design (level 4)

Set the master data rules for the person entity on a talent platform with three sources: the HR system, payroll and the learning platform.

Options: Only when national insurance number and date of birth agree, When name and birth year agree.

1. Automatic merge
2. Uncertain pairs
3. Source IDs after a merge

Sources

- DMBOK2 chapter 10 describes deterministic and probabilistic matching, referring candidate duplicates to data stewards, match-merge and match-link, survivorship rules (most trusted source, most recent, most complete), golden records, and cross-reference management between source IDs and a global ID. (verified: DAMA-DMBOK2, checked in the printed book by the course owner, 7 November 2026.)
- Automatic merging in score bands, with only deterministic keys merging automatically, is this lesson's recommended practice. (unverified: General master data management practice, not a DMBOK2 quotation.)

E2-L5 A RACI and a business glossary for talent data

Concept

Lesson E2-L3 named who decides. A **RACI** turns that into a table people can check, and a **business glossary** records what they decided.

RACI. Rows are activities, columns are roles, and each cell holds one letter:

- **R, responsible**: does the work. At least one per activity.
- **A, accountable**: owns the outcome and signs it off. **Exactly one** per activity.
- **C, consulted**: asked before the decision, a two-way conversation.
- **I, informed**: told after it, one way.

Two common failures: an activity with two As, so each waits for the other, and one with no A, so nobody signs off. Name roles, not people, so the table survives a reorganisation.

Business glossary. One entry per business term, such as "active employee" or "regretted leaver". Each entry holds:

- the **term** and a plain **definition**;
- the **calculation rule**, where it is a measure, such as which contract types count;
- the **owner** (the A for its definition) and the **steward** who maintains it;
- the **system of record** that holds the data;
- **related terms** and the ones it must not be confused with;
- a **status** (draft or approved), a version and the date approved.

A definition changes only through the escalation path in lesson E2-L3, and every change gets a new version, so a report can say which definition it used.

DMBOK2 places the business glossary in **metadata management**. Start with the 15 terms that appear most in people reports, not the whole vocabulary.

Why it matters

Two HR directors each believed they owned the definition of "headcount", and the RACI listed both as accountable. Each approved a different version for their own reports, so the board pack showed two headcounts 4% apart. Making one of them accountable, and recording the approved definition in the glossary with a version, ended it.

Worked example: weak code

```
1  # RACI, talent data
2  define headcount: {HR director: A, People analytics lead: A, Data
     steward: R}
3  grant access to ratings: {Platform team: R}
4  glossary: a shared page where anyone adds terms
```

1. **Line 2.** Two accountable roles. Each will approve their own definition.
2. **Line 3.** Nobody is accountable for who sees performance ratings.
3. **Line 4.** No owner, no status, no version. Nobody knows which definition is approved.

Worked example: strong code

```
1  # RACI, talent data
2  define headcount: {Workforce data owner: A, Data steward: R, Finance: C,
     Report owners: I}
3  grant access to ratings: {Performance data owner: A, Platform team: R,
     DPO: C}
4  # Glossary entry
5  term: Headcount
6  definition: People with an active employment contract on the reporting
     date
7  rule:
     count contracts of type permanent or fixed-term; exclude contractors
     and agency staff
8  owner: Workforce data owner
9  steward: Workforce data steward
10 system_of_record: HR system, employment table
11 not_to_confuse_with: [FTE, contractor count]
```

```
❸ status: approved, version 2, 2026-11-07
```

1. **Line 2.** One A, at least one R, and the people consulted and informed are named.
2. **Line 7.** The calculation rule settles the dispute a definition alone leaves open.
3. **Line 12.** A version and a date let a report say which definition it used.

Give every activity exactly one A and at least one R, name roles rather than people, and keep each glossary term with a rule, an owner, a steward, a system of record and a versioned status.

Recap card

- **RACI letters** R does the work, A owns and signs off (exactly one), C is asked before, I is told after.
- **Two RACI failures** Two accountable roles, so each waits or approves their own; no accountable role, so nobody signs off.
- **Glossary entry fields** Term, definition, calculation rule, owner, steward, system of record, related terms, status with version and date.
- **Where to start a glossary** The 15 terms used most in people reports, not the whole vocabulary.

Practice

Exercise 1. Multiple choice (level 1)

How many accountable roles should each activity in a RACI have?

- a) Exactly one
- b) At least one
- c) One per team involved
- d) None, if someone is responsible

Exercise 2. Multiple choice (level 2)

The data protection officer must give a view before ratings access is granted. Which letter fits the DPO?

- a) C
- b) I
- c) A
- d) R

Why?

- r1) Consulted means asked before the decision, in a two-way exchange.

- r2) Consulted means told after the decision.

Exercise 3. Fill the gap (level 2)

Complete the glossary entry fields.

```
term, definition, calculation _____(1)
_____(2) (accountable for the definition) and steward
system of _____(3)
status, version and date approved
```

Exercise 4. Write the function (level 3)

Write `raci_problems(raci: dict[str, dict[str, str]]) -> list[str]`. `raci` maps each activity to a dict of role to letter.

Return a sorted list of problems, using exactly these messages:

- `"<activity>: no accountable role"` when no role has `"A"`;
- `"<activity>: <n> accountable roles"` when more than one has `"A"`;
- `"<activity>: no responsible role"` when no role has `"R"`;
- `"<activity>: unknown letter <x> for <role>"` for any letter other than R, A, C or I.

Return an empty list when every activity is sound.

What you need to know.

`list.count(x)` counts how many items equal `x`:

```
grades = ["pass", "fail", "pass"]
grades.count("pass")   # 2
```

`dict.values()` gives the values of a dict, and `list(d.values())` turns them into a list.

Your code must pass these checks:

- A sound activity has no problems.
- Report a missing accountable role, and more than one, with the exact messages.
- Report an activity with no responsible role.
- Report any letter other than R, A, C or I.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
VALID_LETTERS = ...

# Problems in a RACI: missing or duplicate accountable roles, no
    responsible role, bad letters.
```

```
def raci_problems(
    raci: dict[str, dict[str, str]],
) -> list[str]: ...
```

Starter code:

```
def raci_problems(raci):
    return []
```

Exercise 5. Architecture design (level 4)

Start the glossary and RACI for the talent platform's first month.

Options: The 15 terms used most in people reports, Every term in the HR system, at once.

1. Which terms first
2. RACI columns
3. Changing a definition

Sources

- DMBOK2 places the business glossary within metadata management. (search-only: DMBOK2 chapter 12 as summarised in secondary sources; not part of the extracts checked in the book.)
- RACI conventions (exactly one accountable, at least one responsible, consulted before, informed after) and the glossary fields listed are common practice. (unverified: General governance practice, not a quotation from a standard.)

Track E best-practice checklist

☐ Treat every talent attribute as personal data. Flag special category data, including what a model infers.

☐ Every use of personal data has a lawful basis. Every use of special category data also has an Article 9 condition.

☐ Employers rarely rely on consent. Record a legitimate interests assessment when you use legitimate interests.

☐ Keep an appropriate policy document for the employment and substantial public interest conditions.

☐ Screen every new use of people data for a DPIA. Large-scale special category data always needs one.

☐ Govern with a frame everyone shares. Map each knowledge area to the talent domains, with an owner.

☐ Score maturity with evidence, per knowledge area and domain. Never average a gap away.

☐ Run the three-step test on every flow that leaves the UK, access included. Adequacy first, then a safeguard with a transfer risk assessment.

☐ Consult works councils and workers before go-live wherever a tool monitors people. The rules differ by country.

☐ The legal team and the data protection officer decide on compliance. These lessons teach you to ask the right questions.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

E2-L1 The DAMA wheel, eleven knowledge areas around governance

E2-L1 exercise 1

Hints. (1) One area decides and oversees how the others run. (2) It holds authority and control over data assets. (3) Data governance.

Answer. b) Data governance

Data governance sits at the hub and directs the ten other knowledge areas.

Common wrong answers.

- `a`: Architecture is one of the ten spokes.
- `c`: Data quality is a spoke. Governance decides the quality rules and who owns them.
- `d`: Metadata management is a spoke.

E2-L1 exercise 2

Hints. (1) Where is the problem: the report or the records? (2) Which area keeps one golden record per person? (3) Reference and master data.

Answer. b) Reference and master data Because: r1) Master data keeps one agreed record per core entity, such as a person.

Duplicate people are a master data problem. The report only exposes them.

Common wrong answers.

- `a`: The report shows the symptom. The duplicate person records cause it.
- `c`: Documents are not involved. The problem is two records for one person.
- `d`: Storage holds the rows. The rule that decides which rows are the same person belongs to master data.
- `b|r2`: Right area, wrong reason. Reference data is a list of codes. One record per person is master data.

E2-L1 exercise 3

Hints. (1) One is a list of allowed values. (2) The other describes data rather than being data. (3) `Reference`, then `Metadata`.

Answer.

```
Reference data: a controlled list of codes, such as grades or country
    codes
Metadata: data about data, such as definitions, owners and lineage
```

Reference data lists allowed codes. Metadata describes data. Master data is the agreed record itself.

Common wrong answers.

- g1 = `Master`: Master data is the agreed record of a core entity, such as a person. A list of codes is reference data.

E2-L1 exercise 4

Hints. (1) Ask what kind of failure each one is: access, meaning or completeness. (2) Where do agreed definitions live? (3) Completeness is a data quality dimension.

Answer.

- Access to salary is a data security question: masking and roles.
- A shared definition lives in the glossary, part of metadata management.
- Missing managers break a completeness rule, owned by data quality.

Naming the area first sends each problem to an owner who can fix it.

Common wrong answers.

- salary = `quality`: The salaries are correct. The problem is who can see them.
- definition = `architecture`: Layers will not settle a definition. The glossary does.
- missing = `security`: Nothing is exposed. Records are incomplete, which is a quality rule.

E2-L1 exercise 5

Hints. (1) Breadth first, then depth. (2) Who can decide what a field means? (3) Is the wheel about knowledge or about teams?

Answer.

- A full map first shows where the risk sits, not where you are comfortable.

- Named owners per area and domain turn the map into accountability. Gaps are findings.
- The wheel is a checklist. Teams can own several areas.

Map all areas, name owners, and use the wheel to route work, not to reorganise people.

Common wrong answers.

- scope = `one`: Starting where you are comfortable can leave the riskiest area unseen.
- owners = `team`: A platform team cannot own the meaning of HR data. Business owners must.
- use = `org`: The wheel organises knowledge, not people.

E2-L2 A maturity baseline you can defend

E2-L2 exercise 1

Hints. (1) Is there a documented standard? (2) Is it measured? (3) Documented and followed, but not measured.

Answer. b) 3 defined

A documented standard followed across teams is level 3, defined. Measuring it with targets would move it to 4.

Common wrong answers.

- `a`: Initial means ad hoc. A documented standard that every team follows is further along.
- `c`: Managed needs measures and targets. Nobody measures these rules yet.
- `d`: Optimised needs continuous improvement driven by measures.

E2-L2 exercise 2

Hints. (1) What would an average hide? (2) Which score stops an AI pilot in its tracks? (3) Report the minimum and name its area.

Answer. c) 0, with the area that scored it Because: r1) One weak area can sink a domain, and a mean hides it.

Report the weakest area per domain. The other scores still matter for the plan, but the minimum sets the risk.

Common wrong answers.

- `a`: The mean of 3 hides an area with no capability at all.

- `b`: The mode hides the weakest area just as the mean does.
- `d`: A baseline reports where you are today, including the gaps.
- `c|r2`: Right answer, wrong reason. The weakest area can be any of the eleven.

E2-L2 exercise 3

Hints. (1) What must exist before any score? (2) Scores before summaries. (3) Rescoring comes last, on the same scale.

Answer.

1. Agree and version the scale with stakeholders
2. Lay out the grid: knowledge areas by talent domain
3. Score each cell in whole levels, citing evidence
4. Report the weakest area per domain
5. Rescore at a fixed interval on the same scale

Scale, grid, scores with evidence, the weakest area per domain, then rescoring on the same scale.

Common wrong answers.

- Scores mean nothing until the scale is agreed. Agree it first.
- You cannot report the weakest area before you score.

E2-L2 exercise 4

Hints. (1) Validate every score before you pick the minimum. (2) Check `bool` before `int`, because `True` is an `int` in Python. (3) `min(areas.items(), key=lambda pair: (pair[1], pair[0]))` breaks ties by name.

Answer.

```python
MIN_LEVEL = 0
MAX_LEVEL = 5


def weakest_by_domain(scores: dict[str, dict[str, int]]) -> dict[str, tuple[str, int]]:
    """The weakest knowledge area and its score for each talent domain.

    Args:
        scores: Domain to knowledge area to whole-level score.

    Returns:
        Domain to (area, score). Ties go to the area first in alphabetical order.

    Raises:
```

```
        ValueError: If a score is not a whole number from 0 to 5, or a
    domain is empty.
    """
    result: dict[str, tuple[str, int]] = {}
    for domain, areas in scores.items():
        if not areas:
            raise ValueError(f"{domain}: no scores")
        for area, score in areas.items():
            if isinstance(score, bool) or not isinstance(score, int):
                raise ValueError(f"{domain}.{area}: score must be a whole
    number")
            if not MIN_LEVEL <= score <= MAX_LEVEL:
                raise ValueError(f"{domain}.{area}: score {score} is
    outside 0 to 5")
        result[domain] = min(areas.items(), key=lambda pair: (pair[1],
    pair[0]))
    return result
```

Validate first, so a typo such as 2.5 or 6 fails loudly. Then a tuple key picks the lowest score and breaks ties by name, so the result never depends on dict order.

E2-L2 exercise 5

Hints. (1) Which view shows the weakest area? (2) What can an auditor check? (3) What blocks work built on top?

Answer.

- The grid shows where the risk sits. One number cannot.
- Evidence lets anyone check a score, and rescoring use the same proof.
- Targets start with the weakest areas, because they block everything built on top.

Show the grid, cite the evidence, and set targets that fix the weakest areas first.

Common wrong answers.

- headline = `mean`: An average of 3.1 hid a 0 in this lesson's example.
- evidence = `survey`: Perception is not evidence. Cite artefacts.
- next = `pilot`: A high mean can still hide a 0 in security or ownership.

E2-L3 Governance at the hub, decision rights, policies and escalation

E2-L3 exercise 1

Hints. (1) Governance decides. Management does. (2) Which option sets a decision right? (3) Who may approve access.

Answer. b) Deciding who may approve access to salary data

Deciding who holds a decision right is governance. Running, building and granting are management.

Common wrong answers.

- `a`: Running checks is doing the work. Governance sets the thresholds and who owns them.
- `c`: Building a layer is management. Governance sets the rules it must meet.
- `d`: Granting is management. Governance decides who may approve the grant.

E2-L3 exercise 2

Hints. (1) Which one could a test check? (2) Policy says what, standard makes it measurable, procedure says how. (3) The masking rule.

Answer. b) Ratings are masked for every role except HR business partners and the line chain. Because: r1) A standard makes a policy measurable, so you can test whether it holds.

The masking rule is measurable: you can test which roles see unmasked ratings. That makes it a standard.

Common wrong answers.

- `a`: That is the policy: what must be true, and why.
- `c`: That is a procedure: how to do it.
- `d`: That is advice. Nobody can test it.
- `b|r2`: Right answer, wrong reason. A standard is defined by being measurable, not by who signs it.

E2-L3 exercise 3

Hints. (1) The top level states intent. (2) The middle level can be tested. (3) `Policy`, `Standard`, `Procedure`.

Answer.

```
Policy: what must be true, and why. Changes rarely, senior sign-off.
Standard: makes it measurable. Changes when the platform does.
Procedure: how to do it. Changes often.
```

Policy, standard and procedure separate intent, measure and method, so each changes at its own pace.

Common wrong answers.

- g1 = `Standard`; g2 = `Policy`: The policy states the intent. The standard makes it measurable.

E2-L3 exercise 4

Hints. (1) Who gathers the facts? (2) Who decides when owners disagree? (3) Record before you publish.

Answer.

1. The data steward gathers the definitions in use and their impact
2. The data owners of both domains try to agree
3. Without agreement, the governance council decides
4. Record the decision and its reason in the decision log
5. Update the glossary entry and tell the report owners

Steward, owners, council if needed, then the log, then the glossary and the people affected.

Common wrong answers.

- Start at the lowest level that can resolve it. The council is the last resort.
- Log the decision first, so the glossary change points to its reason.

E2-L3 exercise 5

Hints. (1) Who owns the meaning of HR data? (2) Which option can be tested? (3) Which option ends a dispute for good?

Answer.

- The business owns meaning. The platform team builds what the owner decides.
- Three levels of rules make access testable and changeable at the right pace.
- A fixed path with a log ends disputes for good.

Named owners, layered rules and a logged escalation path turn governance from advice into decisions.

Common wrong answers.

- decider = `platform`: Building a table does not give the right to define a rating. That belongs to the business owner.
- rule = `guidance`: Guidance cannot be tested or enforced.
- disputes = `meeting`: Without an owner and a log, the same dispute returns.

E2-L4 People master data, match rules and survivorship

E2-L4 exercise 1

Hints. (1) Survivorship runs after matching. (2) It works field by field. (3) Which value survives.

Answer. a) Which value each field of the golden record takes

Matching finds the records. Survivorship picks each field's value for the golden record.

Common wrong answers.

- `b`: That is the match rule. Survivorship comes after the match.
- `c`: That is access control, a data security question.
- `d`: That is retention.

E2-L4 exercise 2

Hints. (1) Which band holds 0.87? (2) Who decides uncertain pairs? (3) The steward queue.

Answer. a) The pair goes to the data steward queue Because: r1) A score between the bands is uncertain, and a false merge mixes two people's data.

Scores between the two thresholds are uncertain, so a person decides.

Common wrong answers.

- `b`: 0.87 is below the merge band.
- `c`: 0.87 is above the keep-apart band. Someone must decide.
- `d`: Replacing a record loses its source ID and history.
- `a|r2`: Right action, wrong reason. Stewards see only the uncertain band, or the queue would never empty.

E2-L4 exercise 3

Hints. (1) The trusted system wins for legal name. (2) Addresses change over time. (3) `priority`, `recent`, `blank`.

Answer.

```
legal_name:    source priority, the HR system first
home_address:  most recent update wins
any field:     never overwrite a value with a blank
```

Source priority for legal name, most recent for address, and a blank never wins.

Common wrong answers.

- g2 = `complete`: An address changes when someone moves. The latest update is the best guess, so the rule is most recent.

E2-L4 exercise 4

Hints. (1) Check every source against `priority` first. (2) Sort the records by `priority.index` of their source, most trusted first. (3) Take a field only if it is not taken yet and its value `is not None`.

Answer.

```python
def survive(records: list[dict], priority: list[str]) -> dict:
    """The golden record built from matched source records.

    Args:
        records: Matched records, each with a "source" key and field values.
        priority: Source names, most trusted first.

    Returns:
        Each field's value from the most trusted source that has one.

    Raises:
        ValueError: If a record's source is not in priority.
    """
    for record in records:
        if record["source"] not in priority:
            raise ValueError(f"unknown source: {record['source']}")
    golden: dict = {}
    for record in sorted(records, key=lambda r: priority.index(r["source"])):
        for field, value in record.items():
            if field != "source" and field not in golden and value is not None:
                golden[field] = value
    return golden
```

Sorting by trust and taking the first non-missing value per field applies source priority while keeping blanks from winning. `is None` keeps 0 as a real value.

E2-L4 exercise 5

Hints. (1) Which rule cannot collide for two people? (2) Who should decide a doubtful pair? (3) How would you undo a false merge?

Answer.

- Automatic merges need a deterministic key. Name and birth year collide in a large firm.
- A steward decides the uncertain band, because a false merge exposes one person's data to another.
- The cross-reference lets later deliveries find their person and makes an unmerge possible.

Deterministic auto-merge, steward review for the middle band, and a kept cross-reference make the golden record safe to use.

Common wrong answers.

- auto = `name`: Two people often share a name and birth year. That rule caused a reportable breach in this lesson's example.
- uncertain = `merge`: A false merge mixes two people's salaries and ratings. Complaints arrive after the harm.
- xref = `drop`: Without source IDs, the next payroll delivery cannot find its person, and you cannot undo a false merge.

E2-L5 A RACI and a business glossary for talent data

E2-L5 exercise 1

Hints. (1) A owns the outcome. (2) What happens with two owners? (3) Exactly one.

Answer. a) Exactly one

Exactly one accountable role per activity, so one person signs off.

Common wrong answers.

- `b`: Two accountable roles wait for each other or approve different things. Exactly one.
- `c`: Teams can be responsible or consulted. Only one role owns the outcome.
- `d`: Without an A, nobody signs the work off.

E2-L5 exercise 2

Hints. (1) Before or after the decision? (2) One-way or two-way? (3) Consulted.

Answer. a) C Because: r1) Consulted means asked before the decision, in a two-way exchange.

C is asked before the decision; I is told after it.

Common wrong answers.

- `b`: Informed is told after the decision. The DPO's view is needed before it.
- `c`: The performance data owner is accountable for access. The DPO advises.
- `d`: Responsible does the work of granting. The DPO does not.
- `a|r2`: Right letter, wrong reason. Being told afterwards is I. C is asked before.

E2-L5 exercise 3

Hints. (1) How a measure is computed. (2) The A for the definition. (3) `rule`, `owner`, `record`.

Answer.

```
term, definition, calculation rule
owner (accountable for the definition) and steward
system of record
status, version and date approved
```

A calculation rule, an owner and a system of record make a definition usable and checkable.

Common wrong answers.

- g2 = `steward`: The steward maintains the entry. The owner is accountable for the definition.

E2-L5 exercise 4

Hints. (1) For each activity, count the As and the Rs in its letters. (2) Check every letter against R, A, C and I. (3) Collect the messages and return them sorted.

Answer.

```python
VALID_LETTERS = frozenset({"R", "A", "C", "I"})


def raci_problems(raci: dict[str, dict[str, str]]) -> list[str]:
```

```
    """Problems in a RACI: missing or duplicate accountable roles, no
responsible role, bad letters.

    Args:
        raci: Activity to a dict of role to letter.

    Returns:
        The problems found, sorted. Empty when the RACI is sound.
    """
    problems: list[str] = []
    for activity, cells in raci.items():
        letters = list(cells.values())
        accountable = letters.count("A")
        if accountable == 0:
            problems.append(f"{activity}: no accountable role")
        elif accountable > 1:
            problems.append(f"{activity}: {accountable} accountable
roles")
        if letters.count("R") == 0:
            problems.append(f"{activity}: no responsible role")
        for role, letter in cells.items():
            if letter not in VALID_LETTERS:
                problems.append(f"{activity}: unknown letter {letter} for
{role}")
    return sorted(problems)
```

Counting As and Rs per activity catches the two failures that stall decisions, and checking letters catches typos before the table is published.

E2-L5 exercise 5

Hints. (1) Where do disputes happen most? (2) What happens when someone leaves? (3) How would a report know the definition changed?

Answer.

- The most-used terms cause most disputes, so they repay the effort first.
- Roles survive reorganisations; names go stale.
- Versioned changes let every report say which definition it used.

Start with the busiest terms, name roles, and version every change.

Common wrong answers.

- scope = `all`: Hundreds of terms at once means none get approved. Start where the disputes are.
- names = `people`: When someone moves on, a name-based RACI points nowhere.
- change = `edit`: An edit in place silently changes every report that uses the term.